Software Requirements Specification

For

CareerOS

An AI-Powered Career Intelligence Platform

Name: Vaibhav Sahu
Roll No.: 25225026

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for CareerOS, an AI-powered career intelligence and employability platform developed as an MCA capstone project. It is intended to serve as the authoritative reference for design, implementation, testing, and evaluation of the system, and as the contract of understanding between the developer and the project evaluators/guide.

1.2 Scope

CareerOS is a unified, AI-driven platform that supports students throughout their academic and early-career journey — from career profiling and resume analysis through skill-gap identification, personalized learning roadmaps, mock interviews, GitHub portfolio analysis, and placement readiness tracking. Unlike disconnected point tools, CareerOS maintains continuous, long-term knowledge of each student's career journey (a "Career Memory") and uses it to personalize recommendations over time.

The platform serves five user roles — Student, Recruiter, Placement Officer, Faculty, and Administrator — each through a dedicated, role-based dashboard. The system is delivered as a web application with a Node.js/Express backend, a Python/FastAPI AI microservice layer, and a Next.js/React frontend, deployed via containerized infrastructure on a cloud platform.

This document covers Release 1 (MVP) scope as agreed for the one-semester capstone timeline; features explicitly marked "Future Scope" are documented for completeness but are out of scope for the graded deliverable.

1.3 Definitions, Acronyms, and Abbreviations

Table 2: Definitions, Acronyms, and Abbreviations

Term	Definition
ATS	Applicant Tracking System — software used by recruiters to filter and rank resumes
SRS	Software Requirements Specification
RAG	Retrieval-Augmented Generation — an AI technique combining retrieval of relevant documents with LLM generation
LLM	Large Language Model
JWT	JSON Web Token, used for stateless authentication
RBAC	Role-Based Access Control
MVP	Minimum Viable Product
SDG	Sustainable Development Goal (United Nations)
Career Memory	The persistent, structured record of a student's skills, activities, and interactions that the AI layer uses to personalize recommendations over time

Term	Definition
Readiness Score	A composite, explainable score indicating a student's placement/employability readiness

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering
- United Nations Sustainable Development Goals (SDG 4: Quality Education, SDG 8: Decent Work and Economic Growth)
- CareerOS Project Proposal (companion document)

1.5 Overview

Section 2 describes the product at a high level — its context, functions, user classes, and constraints. Section 3 details functional requirements organized by module. Section 4 specifies external interface requirements. Section 5 specifies non-functional requirements. Section 6 summarizes the key system models. Section 7 contains appendices, including assumptions, dependencies, and the requirements traceability matrix.

2. Overall Description

2.1 Product Perspective

CareerOS is a new, self-contained product; it is not a modification of an existing system. It replaces a fragmented workflow in which students use separate tools for resume building, interview preparation, learning-path discovery, and portfolio tracking. Conceptually, CareerOS sits between the student's raw career artifacts (resume, GitHub profile, certificates) and the outcomes recruiters and placement cells care about (readiness, fit, skill coverage), using an AI services layer to bridge the two.

The system architecture consists of three cooperating tiers: a React/Next.js frontend, a Node.js/Express API and business-logic backend, and a Python/FastAPI AI microservice layer (resume parsing, embeddings, RAG, LLM orchestration via LangChain) that the backend calls over an internal API. PostgreSQL is the system of record; Redis supports caching and session/queue needs; a vector database (ChromaDB or Pinecone) supports semantic search and RAG retrieval for the Career Memory and AI Mentor features.

2.2 Product Functions

At a high level, CareerOS provides:

- Role-based authentication and dashboards for Student, Recruiter, Placement Officer, Faculty, and Admin
- Resume upload, parsing, ATS scoring, and structured feedback
- Skill gap analysis against a target role, with a personalized learning roadmap
- An AI Career Mentor that retains context across sessions (Career Memory) and gives personalized guidance
- Mock interview simulation with AI-generated feedback
- GitHub portfolio analysis and project recommendations
- Certificate management and a composite, explainable Career Readiness Score
- Recruiter-side job posting, candidate search, and AI-assisted candidate ranking
- Placement cell and faculty analytics on student readiness and department performance
- Administrative user, AI-service, and system management

2.3 User Classes and Characteristics

Table 3: User Classes and Characteristics

User Class	Characteristics
Student	Primary user; typically an undergraduate/postgraduate student with basic digital literacy; uses the platform iteratively over months to build career readiness
Recruiter	Represents a hiring organization; needs efficient search, filtering, and comparison of candidates; moderate technical proficiency
Placement Officer	Institutional staff responsible for placement outcomes; needs aggregate analytics and readiness reporting across cohorts

User Class	Characteristics
Faculty	Mentors/monitors a subset of students; needs visibility into skill tracking and performance, not full admin control
Administrator	Manages platform configuration, users, AI service health, and system logs; highest privilege level

2.4 Operating Environment

- Client: modern evergreen web browsers (Chrome, Edge, Firefox, Safari), responsive design for desktop and tablet
- Server: containerized (Docker) services deployed on a cloud provider (AWS/Azure/GCP)
- Database: PostgreSQL (relational data), Redis (cache/queues), ChromaDB or Pinecone (vector store)
- AI services: Python/FastAPI, LangChain, hosted LLM API (OpenAI or Gemini)

2.5 Design and Implementation Constraints

- Must be deliverable and demonstrable within a single MCA semester timeline
- Must rely on third-party LLM APIs rather than self-hosted models, given compute constraints
- Must follow role-based access control; no user may access another role's dashboard or data
- Must maintain weekly Git commit history for academic evaluation
- Should align with UN SDG 4 (Quality Education) and SDG 8 (Decent Work and Economic Growth)

2.6 Assumptions and Dependencies

- Users have a functioning internet connection and a modern browser
- A third-party LLM API (OpenAI or Gemini) remains available and within free/academic usage tiers for the project duration
- Resume inputs are primarily in PDF/DOCX format and in English
- GitHub portfolio analysis assumes public repositories or an authorized OAuth connection
- Institutional data (placement records, department structure) is provided or mocked for demonstration purposes

3. System Features (Functional Requirements)

Each functional requirement is uniquely identified (module prefix + number) for traceability, and tagged with priority: Must Have (MVP-critical), Should Have (targeted if time allows), or Could Have (stretch/future scope).

3.1 Authentication & User Management (AUTH)

Covers registration, login, role assignment, and session management for all user classes.

Table 4: Authentication & User Management (AUTH) — Functional Requirements

Req ID	Requirement Description	Priority
AUTH-01	The system shall allow a new Student to register using email and password, or via institutional SSO if available.	Must Have
AUTH-02	The system shall authenticate users via JWT-based sessions with configurable expiry.	Must Have
AUTH-03	The system shall enforce role-based access control (RBAC) so each user only accesses their role's dashboard and permitted data.	Must Have
AUTH-04	The system shall allow Admins to create, deactivate, or reassign roles for any user account.	Must Have
AUTH-05	The system shall support password reset via a verified email link.	Should Have
AUTH-06	The system shall log all authentication events (login, logout, failed attempts) for audit purposes.	Should Have

3.2 Student Module — Career Profile & Resume (STU)

Covers the student's core profile, resume upload, parsing, and ATS scoring.

Table 5: Student Module — Career Profile & Resume (STU) — Functional Requirements

Req ID	Requirement Description	Priority
STU-01	The system shall allow a Student to create and edit a Career Profile (education, skills, target roles, interests).	Must Have
STU-02	The system shall allow a Student to upload a resume in PDF or DOCX format.	Must Have
STU-03	The AI service shall parse the uploaded resume into structured fields (skills, experience, education, projects).	Must Have
STU-04	The system shall compute an ATS compatibility score for the uploaded resume with a breakdown of contributing factors.	Must Have
STU-05	The system shall provide actionable, section-by-section feedback for improving the resume.	Should Have
STU-06	The system shall allow the Student to maintain multiple resume versions targeted at different roles.	Could Have

3.3 Student Module — Skill Gap & Learning Roadmap (SKL)

Covers analysis of the gap between a student's current skills and a target role, and generation of a personalized roadmap.

Table 6: Student Module — Skill Gap & Learning Roadmap (SKL) — Functional Requirements

Req ID	Requirement Description	Priority
SKL-01	The system shall allow a Student to select or specify a target job role.	Must Have
SKL-02	The AI service shall compare the student's parsed skills against the target role's required skill profile and identify gaps.	Must Have
SKL-03	The system shall generate a personalized, sequenced learning roadmap (courses/resources/milestones) to close identified gaps.	Must Have
SKL-04	The system shall allow the Student to mark roadmap milestones as complete and track progress over time.	Should Have
SKL-05	The system shall recommend hands-on projects aligned with the student's skill gaps and target role.	Should Have

3.4 Student Module — AI Career Mentor & Career Memory (MEN)

Covers the conversational AI mentor and the persistent memory layer that personalizes it over time.

Table 7: Student Module — AI Career Mentor & Career Memory (MEN) — Functional Requirements

Req ID	Requirement Description	Priority
MEN-01	The system shall provide a conversational AI Career Mentor that answers career-related questions.	Must Have
MEN-02	The AI service shall persist a structured Career Memory per student (skills, goals, prior guidance given, feedback outcomes) in the vector store.	Must Have
MEN-03	The AI Mentor shall retrieve relevant Career Memory context (via RAG) before generating a response, so guidance is personalized rather than generic.	Must Have
MEN-04	The system shall allow the Student to view a summarized history of prior mentor interactions and recommendations.	Should Have
MEN-05	The AI Mentor shall be able to answer questions grounded in the student's own uploaded documents (resume, certificates) via document Q&A.	Should Have

3.5 Student Module — Mock Interview (INT)

Covers AI-driven mock interview simulation and feedback.

Table 8: Student Module — Mock Interview (INT) — Functional Requirements

Req ID	Requirement Description	Priority
INT-01	The system shall allow a Student to start a mock interview session for a selected role and difficulty level.	Must Have

Req ID	Requirement Description	Priority
INT-02	The AI service shall generate role-relevant interview questions dynamically.	Must Have
INT-03	The system shall accept the Student's text (and optionally audio-transcribed) answers and generate structured AI feedback (strengths, gaps, suggested improvements).	Must Have
INT-04	The system shall track mock interview performance over multiple sessions to show improvement trends.	Should Have

3.6 Student Module — GitHub Portfolio & Certificates (PORT)

Covers GitHub analysis, certificate tracking, and the composite Readiness Score.

Table 9: Student Module — GitHub Portfolio & Certificates (PORT) — Functional Requirements

Req ID	Requirement Description	Priority
PORT-01	The system shall allow a Student to connect a GitHub profile (via username or OAuth).	Must Have
PORT-02	The AI service shall analyze public repositories for language mix, commit activity, and project complexity signals.	Should Have
PORT-03	The system shall allow a Student to upload and tag certificates/credentials.	Must Have
PORT-04	The system shall compute a composite Career Readiness Score from resume quality, skill-gap closure, interview performance, and portfolio signals, with a visible breakdown of contributing factors.	Must Have
PORT-05	The system shall display Career Analytics (progress trends, readiness history) to the Student on a dashboard.	Should Have

3.7 Recruiter Module (REC)

Covers job posting, candidate search, and AI-assisted ranking for the Recruiter role.

Table 10: Recruiter Module (REC) — Functional Requirements

Req ID	Requirement Description	Priority
REC-01	The system shall allow a Recruiter to create, edit, and close job postings with required skills and criteria.	Must Have
REC-02	The system shall allow a Recruiter to search and filter candidates by skill, readiness score, and other profile fields.	Must Have
REC-03	The AI service shall rank candidates against a given job posting based on skill-fit and readiness signals, with an explainable rationale.	Should Have
REC-04	The system shall allow a Recruiter to compare multiple shortlisted candidates side by side.	Could Have

3.8 Placement Cell Module (PLC)

Covers cohort-level readiness and placement analytics for institutional placement staff.

Table 11: Placement Cell Module (PLC) — Functional Requirements

Req ID	Requirement Description	Priority
PLC-01	The system shall provide a Placement Officer with a dashboard summarizing student readiness across the institution or department.	Must Have
PLC-02	The system shall generate skill-gap reports aggregated across a cohort or department.	Should Have
PLC-03	The system shall provide placement analytics (offers, readiness trends) exportable for reporting.	Should Have

3.9 Faculty Module (FAC)

Covers faculty visibility into assigned students' progress.

Table 12: Faculty Module (FAC) — Functional Requirements

Req ID	Requirement Description	Priority
FAC-01	The system shall allow Faculty to view the profiles and progress of students assigned to them.	Must Have
FAC-02	The system shall provide Faculty with performance and skill-tracking reports for their assigned students.	Should Have

3.10 Admin Module (ADM)

Covers platform administration, AI service management, and system monitoring.

Table 13: Admin Module (ADM) — Functional Requirements

Req ID	Requirement Description	Priority
ADM-01	The system shall allow an Admin to manage (create/edit/deactivate) user accounts across all roles.	Must Have
ADM-02	The system shall allow an Admin to monitor AI service health (latency, error rate, API usage/cost) from a dashboard.	Should Have
ADM-03	The system shall provide an Admin with access to system logs and audit trails.	Should Have
ADM-04	The system shall allow an Admin to generate platform-wide usage reports.	Could Have

4. External Interface Requirements

4.1 User Interfaces

- A responsive web UI (Next.js/React/TypeScript/Tailwind) with distinct dashboards for each of the five user roles.
- Consistent navigation, accessible color contrast, and support for both desktop and tablet viewports.
- Key screens: Login/Register, Student Dashboard, Resume Upload & Analysis, Skill Gap & Roadmap, AI Mentor Chat, Mock Interview, Portfolio & Certificates, Recruiter Search & Ranking, Placement Analytics, Faculty View, Admin Console.

4.2 Hardware Interfaces

None beyond standard client devices (desktop/laptop/tablet) capable of running a modern web browser; no specialized hardware is required.

4.3 Software Interfaces

Table 14: Software Interfaces

Interface	Purpose
LLM Provider API (OpenAI or Gemini)	Used by the AI microservice for resume parsing assistance, mentor conversation, interview question/feedback generation, and roadmap generation
GitHub API	Used for portfolio analysis (repository metadata, languages, commit activity)
PostgreSQL	System of record for users, profiles, jobs, applications, and structured career data
Redis	Caching, session storage, and background job queues
ChromaDB / Pinecone	Vector store for Career Memory embeddings and RAG retrieval
SMTP/Email provider	Transactional email for verification and password reset

4.4 Communication Interfaces

- HTTPS/REST between the frontend and the Node.js/Express backend.
- Internal REST calls between the Express backend and the Python/FastAPI AI microservice.
- Optional WebSocket channel for streaming AI Mentor and mock-interview responses.

5. Non-Functional Requirements

5.1 Performance

- The system shall return standard (non-AI) API responses within 500 ms under normal load.
- AI-backed responses (resume parsing, mentor chat) shall begin streaming or return within 5 seconds under normal load.
- The system shall support at least 100 concurrent users in the demonstration deployment without degradation.

5.2 Security

- All traffic shall be encrypted in transit via HTTPS/TLS.
- Passwords shall be stored using a salted, industry-standard hash (e.g., bcrypt/argon2).
- Access to any resource shall be authorized via RBAC checks on every request, not solely enforced in the UI.
- Uploaded documents (resumes, certificates) shall be scanned for type/size limits before processing.
- Personally identifiable student data shall not be sent to third-party LLM APIs beyond what is functionally necessary, and shall be documented in the privacy notes.

5.3 Reliability & Availability

- Core student-facing features (profile, resume, roadmap) shall degrade gracefully if the AI service is temporarily unavailable, rather than failing the entire request.
- The system shall target 99% uptime during the evaluation/demo period.

5.4 Scalability

- The backend and AI microservice shall be independently deployable and horizontally scalable via containerization (Docker).
- The vector store and relational database shall be selected/configured to support growth in student and document volume without redesign.

5.5 Usability

- A first-time Student user shall be able to complete profile creation and a first resume analysis without external instructions, within a 10-minute session.
- Error messages shall be specific and actionable rather than generic failure codes.

5.6 Maintainability

- The codebase shall follow a modular, layered architecture (controllers/services/repositories) with clear separation between the Node backend and the Python AI services.
- The system shall maintain automated tests for core business logic and critical AI-service integration points.

5.7 Portability

- The system shall be deployable via Docker to any of AWS, Azure, or GCP without code changes, using environment-based configuration.

6. System Models

6.1 Use Case Summary

The primary use cases per role are summarized below; detailed use-case diagrams and sequence/activity diagrams are provided in the companion UML Diagrams deliverable.

Table 15: Use Case Summary per Actor

Actor	Key Use Cases
Student	Register/Login; Manage Career Profile; Upload & Analyze Resume; View Skill Gap; Follow Learning Roadmap; Chat with AI Mentor; Take Mock Interview; Connect GitHub Portfolio; Manage Certificates; View Readiness Score
Recruiter	Register/Login; Post Job; Search Candidates; View AI Ranking; Compare Candidates
Placement Officer	Login; View Cohort Readiness Dashboard; View Skill Gap Reports; View Placement Analytics
Faculty	Login; View Assigned Students; View Performance Reports
Administrator	Login; Manage Users; Monitor AI Services; View System Logs; Generate Reports

6.2 High-Level Data Flow

A Student's resume and profile data flow from the frontend to the Express backend, which persists structured data in PostgreSQL and forwards documents to the FastAPI AI service. The AI service parses, embeds, and stores relevant vectors in the vector database, then returns structured results (parsed fields, ATS score, skill gaps) to the backend, which relays them to the frontend and updates the student's Career Readiness Score. Subsequent AI Mentor and mock-interview interactions retrieve this stored context via RAG to personalize responses.

6.3 Requirements Traceability (excerpt)

A full requirements traceability matrix mapping each requirement ID to design components and test cases is maintained as a companion spreadsheet and updated throughout development. An excerpt is shown below.

Table 16: Requirements Traceability Matrix (Excerpt)

Req ID	Design Component → Verification
STU-03	Resume parsing service (FastAPI) → verified via unit tests on sample resumes
SKL-02	Skill Gap Engine → verified via test cases comparing known profiles to target roles
MEN-03	RAG retrieval pipeline (LangChain + vector store) → verified via mentor response relevance tests
PORT-04	Readiness Score aggregator → verified via unit tests on scoring formula

7. Appendices

7.1 Alignment with UN Sustainable Development Goals

- SDG 4 (Quality Education): personalized learning roadmaps close skill gaps and improve access to relevant learning resources.
- SDG 8 (Decent Work and Economic Growth): improved placement readiness and recruiter-candidate matching support better employment outcomes for students.

7.2 Future Scope

- Multi-institution/tenant support for the Admin module.
- Audio/video-based mock interview analysis (tone, pacing, facial cues).
- Candidate comparison and bulk-shortlisting tools for Recruiters (Could Have, may be included if time permits).
- Mobile native applications.

7.3 Open Issues

- Final choice between OpenAI and Gemini as the primary LLM provider, pending cost/latency evaluation.
- Final choice between ChromaDB (self-hosted, free) and Pinecone (managed, usage-based) for the vector store.
- Scope of GitHub OAuth vs. public-username-only analysis for the MVP.